

Low-Level Design (LLD)

1. Module-Level Design

- **Document Processing Module:** Uses PyPDFLoader to stream PDF content into raw text blocks, preserving metadata like page numbers for source tracking.
- **Chunking Module:** Implements the RecursiveCharacterTextSplitter. It splits text by paragraphs (`\n\n`), then sentences (`\n`), then words, ensuring semantic units remain intact.
- **Embedding Module:** Utilizes HuggingFaceEmbeddings with the all-MiniLM-L6-v2 model to transform text chunks into 384-dimensional dense vectors.
- **Vector Storage Module:** Managed by **ChromaDB**. It handles the initialization of the local `./chroma_db` directory and the indexing of vector-text pairs.
- **Retrieval Module:** Uses a **Vector Store Retriever** with a similarity search algorithm (Cosine Similarity) to fetch the top `$K=3$` relevant chunks for any given query vector.
- **Query Processing Module:** Acts as the entry point, sanitizing user input and passing it to the Graph Execution engine.
- **Graph Execution Module:** Built on **LangGraph**, it orchestrates the StateGraph, controlling the movement of data between nodes based on logic gates.
- **HITL Module:** A specialized node that suspends the automated flow to capture terminal-based input from a human supervisor.

2. Data Structures

Document & Chunk Representation

JSON

```
{
  "page_content": "The return policy is 30 days...",
  "metadata": {
    "source": "manual.pdf",
    "page": 12
  }
}
```

State Object (Graph Schema)

The AgentState is a TypedDict that serves as the single source of truth during the execution lifecycle:

- query: (str) The user's input.
- response: (str) The final output string.
- context: (str) The aggregated text from retrieved chunks.
- next_action: (str) A routing flag ("human" or "ai").

3. Workflow Design (LangGraph)

The graph consists of specific nodes and directed edges:

- **Nodes:** * router: Analyzes intent.
 - rag_node: Executes retrieval and generation.
 - human_node: Captures manual intervention.
- **Edges:**
 - **Conditional Edge:** Connects router to either rag_node or human_node.
 - **Direct Edge:** Connects both processing nodes to END.

4. Conditional Routing Logic

The routing engine applies the following criteria to determine the path:

Scenario	Logic	Destination
Standard Inquiry	General keywords (hours, policy, items).	rag_node
Complex Query	Long sentences or multi-part questions.	human_node
Escalation Trigger	Keywords: "lawsuit", "refund", "manager", "legal".	human_node
Missing Context	<i>If Vector Search score < threshold (e.g., 0.5).</i>	human_node

5. HITL Design

1. **Trigger:** Escalation is triggered by the router when it identifies high-risk keywords.
2. **Process:** The `human_node` function executes a blocking `input()` call in the terminal.
3. **Integration:** The string captured from the human is saved into the response key of the `AgentState`. The graph then proceeds to the END node, bypassing the LLM entirely.

6. API / Interface Design

- **Input Format:** Simple string via CLI.
- **Output Format:** Structured output including the node name and the final response string.
- **Interaction Flow:**
 1. User prompt received.
 2. Graph initialized with input.
 3. Node-by-node execution displayed for transparency.
 4. Final answer rendered to console.

7. Error Handling

- **Missing Data:** If the data/ folder is empty, the system throws a `FileNotFoundError` during the ingestion phase.
- **No Relevant Chunks:** The RAG prompt contains a "Guardrail" instruction: *"If you do not find the answer in the context, say you don't know."*
- **LLM Failure:** Wrapped in a try-except block; if the Groq API fails (Rate Limit/Connection), the graph automatically re-routes to the `human_node` as a fallback.